

0.99.1. Une lettre qui tire — le A bouge, chaque tir garde sa ligne

Le 0.99, avec trois tirs : chaque tir recopie la ligne du A au moment où il part, dans son propre `ty[i]`. Tire, bouge, tire encore : les tirs volent sur des lignes différentes.

1. Ce qu'il faut comprendre

C'est le 0.99, avec plusieurs tirs. Le A bouge avec la croix, et le bouton A lance une rafale. Tire, descends de deux lignes, tire encore : les premiers tirs continuent sur leur ligne, les nouveaux partent sur la nouvelle.

Ce qui est nouveau ici : `ty[3]`, un tableau de lignes. Au 0.99, un tir avait sa ligne, `ty`. Maintenant, chaque tir a la sienne : `ty[i]`. Au départ du tir n° i, `ty[i] = ay` recopie la ligne du A à **ce moment-là**. Ensuite, le tir n'utilise que `ty[i]`.

Le reste ne change pas : `tx[3]` et `voile[3]` (le 0.96.1), la rafale avec `chaque(250)` (le 0.97.1), le A qui bouge et ne marche que sur une case vide (le 0.99).

Déroulons : A en (2, 8). Bouton A : le tir 0 part, `tx[0] = 3`, `ty[0] = 8`. BAS, BAS : le A est en (2, 10). Bouton A : le tir 1 part, `tx[1] = 3`, `ty[1] = 10`. Le tir 0, lui, est toujours sur la ligne 8 : deux tirs, deux lignes.

Essaie : garde A enfoncé et monte ou descends en même temps : une rafale en escalier.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **880 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	66 o	← la console, à l'allumage — la mise en route de la console
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
main()	469 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer(), texte() — trouve l'adresse d'une case de l'écran

EffacerCases	12 o	← effacer()	– efface des cases
EcrireTexte	10 o	← texte()	– écrit un texte, lettre par lettre
CopierTuiles	40 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	32 o	← CopierTuiles	– 2 lettres sur 43
"."	1 o	← texte()	– un texte gravé
"."	1 o	← texte()	– un texte gravé

Lettres gravées : A.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

texte()	× 2	68 o	sur place, dans main()	+ AdresseCase (10 o), EcrireTexte (10 o)
effacer()	× 2	46 o	sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 2	26 o	sur place, dans main()	+ AdresseCase (10 o)
bouton()	× 5	25 o	sur place, dans main()	+ LireManette (37 o)
lire()	× 1	11 o	sur place, dans main()	+ AdresseCase (10 o)
image()	× 1	3 o	sur place, dans main()	+ AttendreImage (28 o)
images()	× 1	2 o	sur place, dans chaque_minuteur()	

Tes fonctions appelées :

chaque_minuteur()	× 3	32 o	sur place, dans main()
-------------------	-----	------	------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : le A bouge – plusieurs tirs ----
// Le 0.99 (le A bouge, chaque tir garde sa ligne), avec trois tirs.
// Chaque tir a SA ligne : un troisième tableau, ty.

#include <poser>          // pose une tuile sur une case du fond
#include <ALPHABET>       // les lettres de la police : ALPHABET[0] est le A
#include <chaque>         // répond 1 toutes les n millisecondes
#include <bouton>         // lit un bouton de la manette
#include <lire>           // lit la tuile posée sur une case
#include <effacer>        // efface des cases, ou tout le fond
#include <texte>          // écrit un texte à l'écran
```

2. UNE VARIABLE — AX

```
uint8_t ax = 2;          // le A : sa colonne...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;          // ...et sa ligne
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;          // la case d'ARRIVÉE du A (le 0.82)
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UN TABLEAU — TX

```
uint8_t tx[3];           // les colonnes des trois tirs...
```

7. UN TABLEAU — TY

```
uint8_t ty[3];           // NOUVEAU : ...et LEURS lignes. Chacun a la sienne.
```

8. UN TABLEAU — VOLE

```
uint8_t vole[3];         // vole[i] = 1 : le tir n° i est en vol
```

9. UNE VARIABLE — LANCE

```
uint8_t lance = 0;
```

10. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

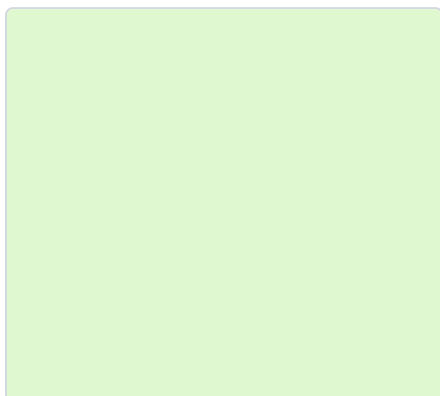
        // ---- le A bouge avec la croix, un pas toutes les 150 ms (le 0.99) ----
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
            } else if (bouton(BAS)) {
                ny = ay + 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- le bouton A lance un tir, au plus toutes les 250 ms ----
        if (chaque(250)) {
            if (bouton(A) && ax < 19) {
                lance = 0;
                for (uint8_t i = 0; i < 3; i++) {
                    if (lance == 0 && vole[i] == 0) {
                        vole[i] = 1;
                        tx[i] = ax + 1;
                        ty[i] = ay;          // NOUVEAU : la ligne du A, recopiée pour CE tir
                        texte(tx[i], ty[i], ".");
                        lance = 1;
                    }
                }
            }
        }

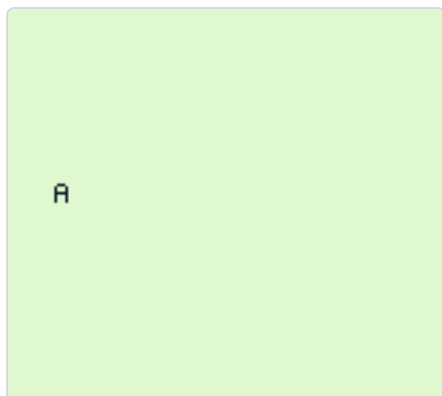
        // ---- chaque tir avance sur SA ligne, ty[i] ----
        // Tire, monte, tire encore : les tirs volent sur des lignes différentes.
        if (chaque(100)) {
            for (uint8_t i = 0; i < 3; i++) {
                if (vole[i] == 1) {
                    effacer(tx[i], ty[i], 1);
                    tx[i] = tx[i] + 1;
                    if (tx[i] < 20) {
                        texte(tx[i], ty[i], ".");
                    } else {
                        vole[i] = 0;
                    }
                }
            }
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



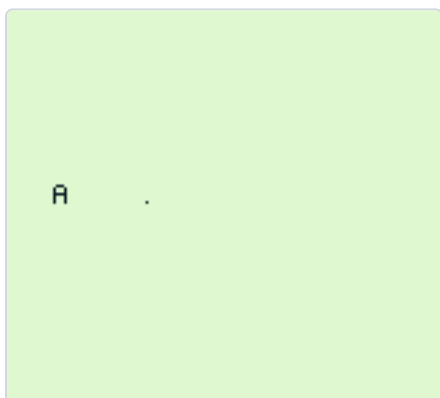
la toute première image — la console vient de démarrer — image 1



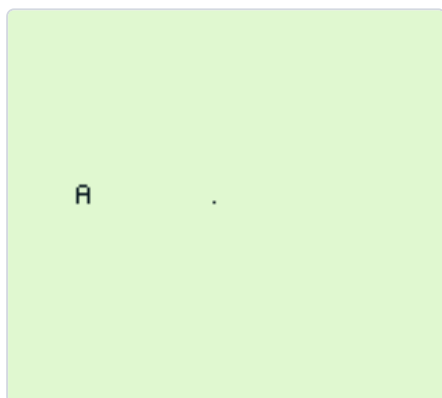
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

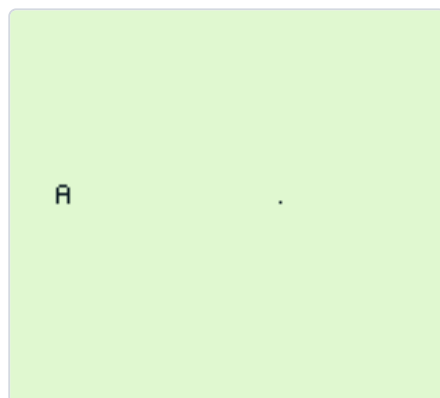
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé



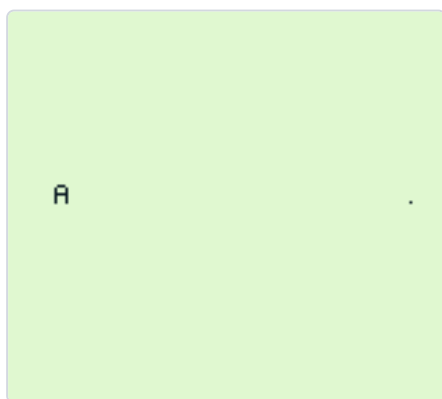
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le A bouge avec la croix et tire des rafales ; chaque « . » continue sur la ligne où il est parti, même quand le A change de ligne.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ un tir sur la ligne 8, puis le A descend en ligne 10 et tire encore

✓ les deux tirs volent en même temps, chacun sur SA ligne (8 et 10)

✓ ils finissent par disparaître au bord

Ensuite : **0.100. Une lettre qui tire — dans le sens où le A regarde** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++